

1. Project Title

Smart Care: Patient Management System.

2. Introduction / Background

To handle patient records, appointments, medications, and communications, healthcare professionals need scalable, secure, and effective solutions. Numerous clinics continue to rely on manual or fragmented digital technologies, which results in poor patient follow-up, missed appointments, and repetitive data entering. HIPAA-conscious (or comparable) patient management systems with real-time functionality, strong authentication, audit trails, and integrated communication channels can be quickly developed thanks to modern web stacks and backend-as-a-service platforms.

The goal of this project is to create SmartCare, a web-based patient management system (PMS) using Next.js + TypeScript on the front end, Tailwind + shadcn/ui for design, Appwrite for backend services (auth, database, storage, functions), Twilio for SMS and OTP notifications, and Sentry for error monitoring. For small-to-medium clinics, the system will simplify appointment scheduling, patient data administration, notifications, and basic reporting.

3. Problem Statement

Paper records or disconnected digital tools (spreadsheets, messaging applications) are used by many private offices and small clinics to handle patients. Conflicts in scheduling, postponed follow-ups, and disjointed medical histories result from this. An integrated, user-friendly, secure online application is required so that administrators and physicians can manage patients, appointments, medications, and communications from a single location while maintaining data protection and dependability.

4. Objectives of the Project

4.1 General Objective:

To create a web-based patient management system that is safe, scalable, and responsive to streamline administrative processes and enhance patient communication.

4.2 Specific Objectives:

- To design and develop a secure, role-based patient management system.

- To implement efficient patient and appointment management functionalities including registration, scheduling, rescheduling, and record maintenance.
- To integrate automated communication mechanisms such as SMS notifications and OTP verification.
- To ensure system reliability and maintainability by incorporating real-time error tracking, logging, and performance monitoring.

5. Scope of the Project

- User authentication and role-based access.
- Patient profile management.
- Appointment booking & calendar.
- SMS notifications for appointment reminders and OTPs.
- Error monitoring.
- Responsive UI with accessibility considerations.

6. Significance of the Project

SmartCare will improve patient care continuity by lowering administrative costs, minimizing missed appointments with automated reminders, and giving physicians easy access to patient records. The project shows how to employ backend-as-a-service patterns with Appwrite, integrate third-party services (Twilio, Sentry), and apply contemporary full-stack development techniques for academic purposes.

7. Literature Review

Cloud-enabled modular solutions that prioritize interoperability, security, and user experience have replaced on-premise monolithic patient management and EMR systems. Research indicates that appointment reminders considerably lower the number of no shows, automated calls and SMS are especially successful. Although they offer advantages for quick development, backend-as-a-service platforms necessitate cautious data governance. Consistent UX is accelerated by UI component libraries and design systems (Tailwind + component systems like shadcn/ui). When managing user data and preserving uptime, observability and error monitoring (Sentry) are essential.

8. Proposed Methodology / System Approach

8.1 Development Model:

Agile Model (Iterative Development)

Reason: Requirements may evolve and improvements can be added incrementally after user testing.

8.2 Tools & Technologies:

- **Languages:** TypeScript
- **Frontend Framework:** Next.js
- **Styling:** Tailwind CSS and shadcn/ui component primitives
- **Backend:** Appwrite
- **Notifications:** Twilio
- **Error Monitoring:** Sentry
- **Optional:** Docker for local Appwrite + services

9. System Requirements

9.1 Hardware Requirements:

- Processor: Minimum Dual-Core
- RAM: 4 GB (8 GB recommended)
- Storage: 2–4 GB development files

9.2 Software Requirements:

- OS: Windows / macOS / Linux
- Node.js (LTS)
- Next.js
- Docker (for local Appwrite)
- Appwrite server or managed instance
- Twilio account

- Sentry account

10. System Architecture / System Design

Will include:

Use Case Diagram:

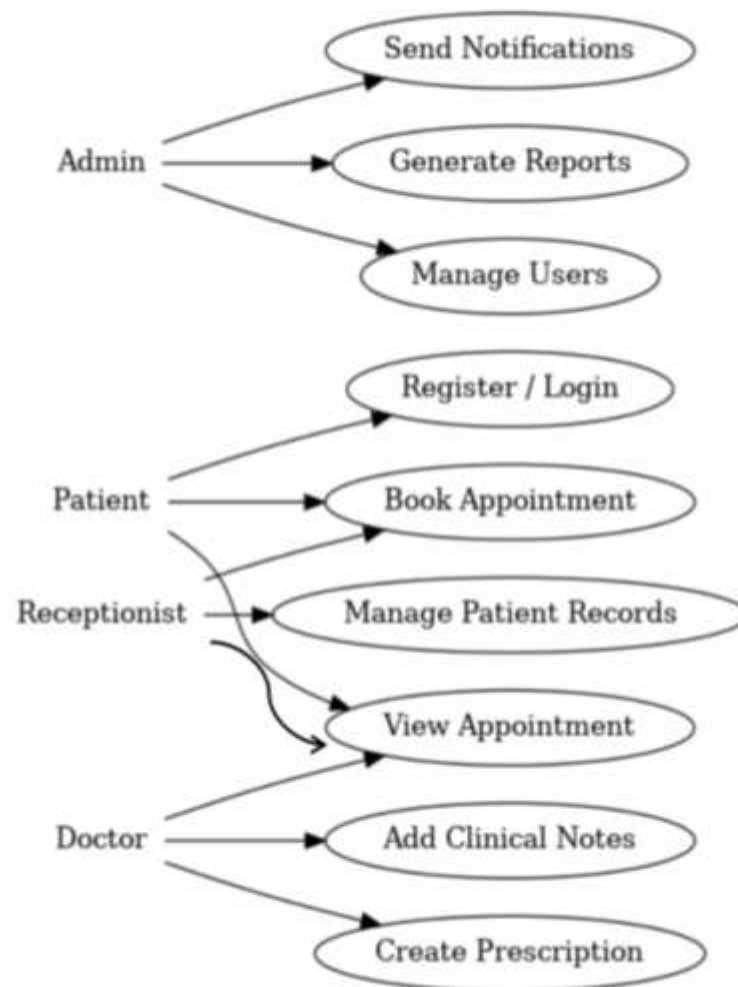


Figure 1: Use Case Diagram

ER Diagram:

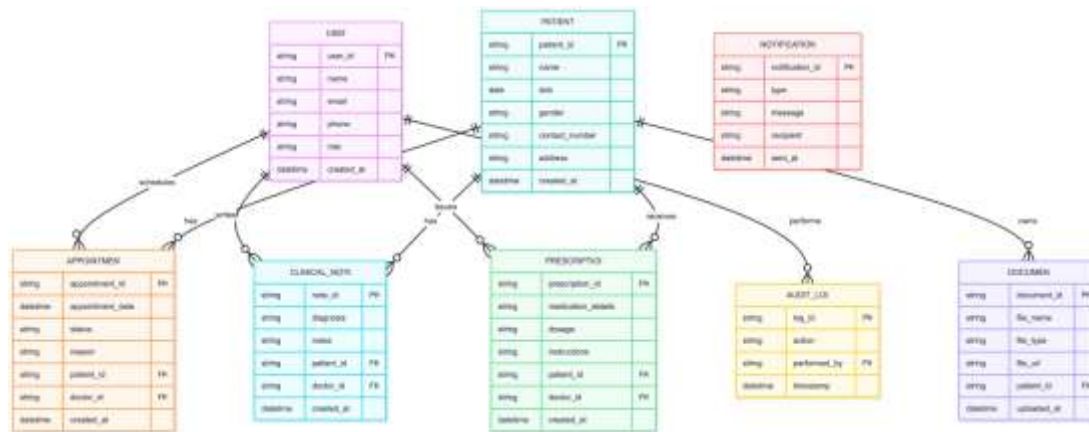


Figure 2: ER Diagram

SYSTEM ARCHITECTURE:

To guarantee scalability, security, and maintainability, the Patient Management System (SmartCare) System Architecture was created utilizing a contemporary full-stack, service-oriented methodology. Next.js and TypeScript are used to build the frontend, which offers a responsive, server-side rendered user interface with shadcn/ui components and Tailwind CSS for accessibility and consistency. Through this web interface, users including patients, physicians, receptionists, and administrators interact with the system. Appwrite Authentication manages all authentication, authorization, and role-based access control, guaranteeing safe user management and login. Appwrite Database is used to store core application data, such as patient records, appointments, prescriptions, and audit logs; Appwrite Storage is used to manage documents like lab reports. Appwrite Functions are used to perform backend business logic and scheduled actions, including sending appointment reminders. They interact with Twilio APIs to provide users with OTPs and SMS notifications. Sentry, which records frontend and backend faults, performance problems, and exceptions in real time and enhances dependability and maintainability, is used to monitor the entire system. Appwrite is either self-hosted or managed, allowing for smooth communication between components and guaranteeing a safe, effective, and scalable patient management solution. The frontend application is put on a cloud platform like Vercel.

11. Modules and Functional Description

1. **Authentication & Authorization Module:** Sign-up/login, role-based access via Appwrite Auth; OTP via Twilio.
2. **Patient Management Module:** Add/update patient profiles, upload attachments to storage.
3. **Appointment Module:** Booking, rescheduling, cancellation, calendar view, availability management.
4. **Clinical Notes & Prescription Module:** Add clinical notes, create prescriptions, download printable prescription.
5. **Notification Module:** Twilio integration for appointment reminders, OTPs, and alerts.
6. **Dashboard & Reports Module:** Daily schedule, no-show report, quick stats for administrators.
7. **Storage & Document Module:** Upload and manage lab reports / attachments.
8. **Audit & Logging Module:** Track actions for accountability; Sentry + Appwrite logs.
9. **Error Monitoring Module:** Sentry integrated in frontend and Appwrite functions for real-time alerts.

12. Expected Output / Deliverables

- Working SmartCare web app (deployed).
- Source code (Next.js + TypeScript).
- Appwrite configuration & schema definitions.
- Integration scripts for Twilio and Sentry.
- System design documentation, API documentation.
- Test cases and testing report.
- Project report and presentation slides.

13. Project Timeline / Gantt Chart

- Week 1–2: Requirement analysis
- Week 3–4: UI design and wireframes
- Week 5–6: AI integration + backend logic

- Week 7–8: Frontend development
- Week 9: Testing and debugging
- Week 10: Refinement and documentation
- Week 11–12: Final deployment and presentation prep

Gantt Chart:

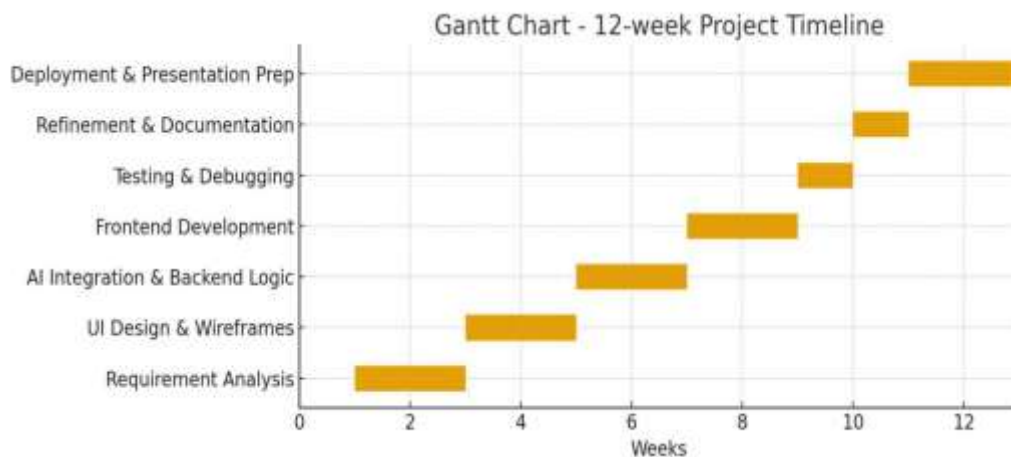


Figure 3: Gantt Chart

14. Limitations

- This initial prototype won't be certified as a full EMR system; regulatory compliance (HIPAA, GDPR specifics) must be implemented for production use.
- Advanced diagnostics, image analysis, or telemedicine features are out of scope.
- Dependence on third-party services (Appwrite, Twilio) means availability is tied to those providers.
- Only English text supported initially.

15. Future Enhancements

- Full EMR features & regulatory compliance workflows
- Payment gateway for billing and invoice payments
- Multi-language UI & localization

16. Conclusion

Using Next.js + TypeScript, Tailwind + shadcn/ui for the frontend, Appwrite for backend services, Twilio for communications, and Sentry for production observability, SmartCare will offer a cutting-edge, safe, and maintainable patient management solution. The project is possible within a 12-week academic timeframe

and strikes a balance between contemporary development patterns and actual clinic needs.

17. References

Next.js Documentation. (2024). Next.js , The React Framework for the Web. Retrieved from <https://nextjs.org/docs>

Appwrite Team. (2024). Appwrite Documentation: Secure Backend Server for Web, Mobile & Flutter Apps. Retrieved from <https://appwrite.io/docs>

Tailwind Labs. (2024). Tailwind CSS Documentation. Retrieved from <https://tailwindcss.com/docs>

shadcn. (2024). shadcn/ui, Re-usable Components Built with Radix UI and Tailwind CSS. Retrieved from <https://ui.shadcn.com>

Twilio Inc. (2024). Twilio Programmable SMS & Verify API Documentation. Retrieved from <https://www.twilio.com/docs>

Sentry. (2024). Sentry Error Tracking and Performance Monitoring Documentation. Retrieved from <https://docs.sentry.io>

TypeScript Team. (2024). TypeScript Handbook. Microsoft. Retrieved from <https://www.typescriptlang.org/docs>

Vercel Inc. (2024). Vercel Deployment Platform Documentation. Retrieved from <https://vercel.com/docs>

Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education Limited.